

ADVANCED OPERATING SYSTEMS

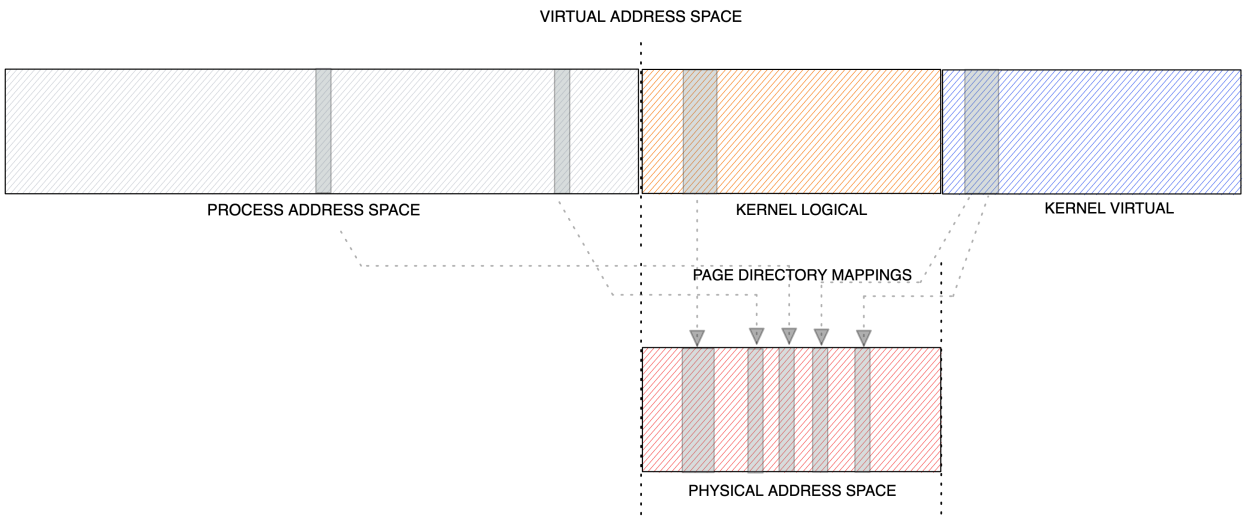
Lab 4 — Kernel allocation and userspace access

Vittorio Zaccaria

Politecnico di Milano

Kernel pages and objects

A1 · Four page mappings



`kmalloc(4 * PAGE_SIZE)` and `vmalloc(4 * PAGE_SIZE)` each return four consecutive **virtual** pages. Predict the PFNs before loading the module.

Module source

A1 · Build and load

```
# Host: start the amd64 guest environment
T=amd64 E=full make dev-up
T=amd64 E=full make dev-build
T=amd64 E=full make dev-run
# Guest
insmod /modules/lab-4-vm-memalloc.ko
dmesg | grep -E 'kmalloc\[|vmalloc\['
```

A1 · Read the PFNs

Selected messages from one amd64 QEMU run (addresses and PFNs vary):

```
kmalloc[0] va=ffff888006110000 pfn=6110
vmalloc[0] va=ffffc90000228000 pfn=6103
kmalloc[1] va=ffff888006111000 pfn=6111
vmalloc[1] va=ffffc90000229000 pfn=6102
kmalloc[3] va=ffff888006113000 pfn=6113
vmalloc[3] va=ffffc9000022b000 pfn=6100
```

`kmalloc` requires consecutive PFNs here; `vmalloc` does not. Both VAs advance by one page. The descending vmalloc PFNs are one observation, not a promised layout.

A2 · SLUB constructor

```
cache allocated=1 constructed=256
```

One `kmem_cache_alloc` call; this run constructed 256 slots when populating a slab. The number of slots is allocator-dependent. The object and cache are freed before module initialization returns.

Crossing the userspace boundary

B1 · Procfs copies

`/proc/lab4` is a root-only teaching endpoint: writes accept at most 16 bytes; reads return the fixed `kernel!\n` reply.

```
uncopied = copy_from_user(text, user, count);
uncopied = copy_to_user(user, reply + *offset, length);
```

B1 · Successful transfers

```
# Guest, while the module is loaded
printf ciao > /proc/lab4
cat /proc/lab4
dmesg | grep -E 'from_user|to_user|received='
```

```
from_user requested=4 uncopied=0
received=ciao
to_user requested=8 uncopied=0
```

`cat` prints `kernel!` ; `uncopied=0` means a complete transfer. Another read on the same file reaches EOF. Exit QEMU with `Ctrl-]` , then `x` , before rebuilding for B2.

B2 · Page-boundary fault

Fault client source: two mapped pages, second set to `PROT_NONE`; each request starts four bytes before the boundary and asks for eight.

```
# Host: compile for the guest after dev-build
docker exec lkp-dev-amd64 gcc -static -O2 -Wall -Wextra -Werror \
  /repo/modules/lab-4-vm-memalloc/user-demo.c \
  -o /staging/initramfs/fs/lab4-user-demo
docker exec -e __BUILD_ARCH=amd64 lkp-dev-amd64 /sources/mini-bootstrap.sh
T=amd64 E=full make dev-run
# Guest: load the new module
insmod /modules/lab-4-vm-memalloc.ko
/lab4-user-demo
echo $?
```

B2 · Interpret the fault

```
from_user requested=8 uncopied=4
to_user requested=8 uncopied=4
write=-1 errno=14; read=-1 errno=14
```

Four accessible bytes were copied; four were not. Both callbacks return `-EFAULT` (userspace `errno=14`) and the client exits `0`. A failed read leaves its file offset unchanged, but may have modified the accessible prefix of the userspace buffer.

```
# Guest cleanup
rmmod module
test ! -e /proc/lab4 && echo removed
```